Lab03_VC_Docker_Tekup

Amen Allah Ben Dhafer _ ING-3-S-D

Introduction à Docker et manipulation des conteneurs – TEK-UP University

Objectif du TP

L'objectif de ce TP est de se familiariser avec **Docker**, la plateforme de conteneurisation d'applications. Les manipulations sont réalisées sur la machine virtuelle **Ubuntu 24.04** sous VirtualBox : installation de Docker, gestion des images et des conteneurs, puis exécution d'un conteneur simple, **hello-world**.

Section	Figures	Contenu
Prérequis et installation	1 – 5	Version du noyau, mise à jour des paquets, installation de docker.io
Post-installation	6 – 8	Service Docker, groupe docker, accès sans sudo
Commande docker	9 – 16	Sous-commandes disponibles et options de docker run
Gestion des images	17 – 26	Lister, chercher, télécharger et supprimer des images
Gestion des conteneurs	27 – 39	Créer, démarrer, arrêter, supprimer, exécuter une commande
Image hello-world	40 – 47	Exécution, anatomie, code source et Dockerfile

Prérequis

Docker nécessite un système Linux 64 bits avec un noyau 3.10 au minimum, des droits d'administration et une connexion Internet active. La version du noyau est vérifiée avec la commande `uname -r`.

```
$ uname -r
```

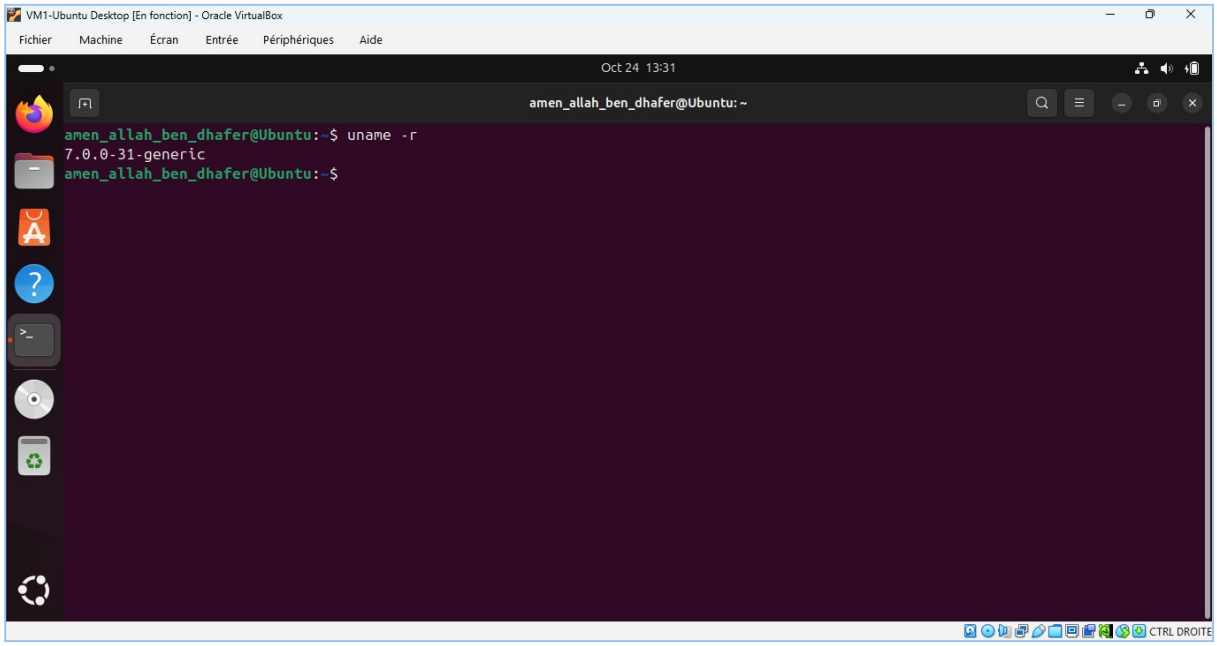


Figure 1 – *Version du noyau Linux de la VM Ubuntu*

> **Observation :** Le noyau installé est le **7.0.0-31-generic**, très supérieur à la version minimale 3.10 : le système répond au prérequis.

1. Installation de Docker sous Ubuntu

Docker peut être installé depuis le dépôt officiel Docker (méthode 1) ou depuis le dépôt officiel d'Ubuntu (méthode 2). Dans ce TP, l'installation est réalisée avec la **méthode 2**, via le paquet `docker.io`.

1.2 Méthode 2 : dépôt officiel Ubuntu

Étape 1 : Mise à jour des paquets

La commande `apt-get update` synchronise l'index des paquets avec les dépôts configurés.

```
$ sudo apt-get update
```

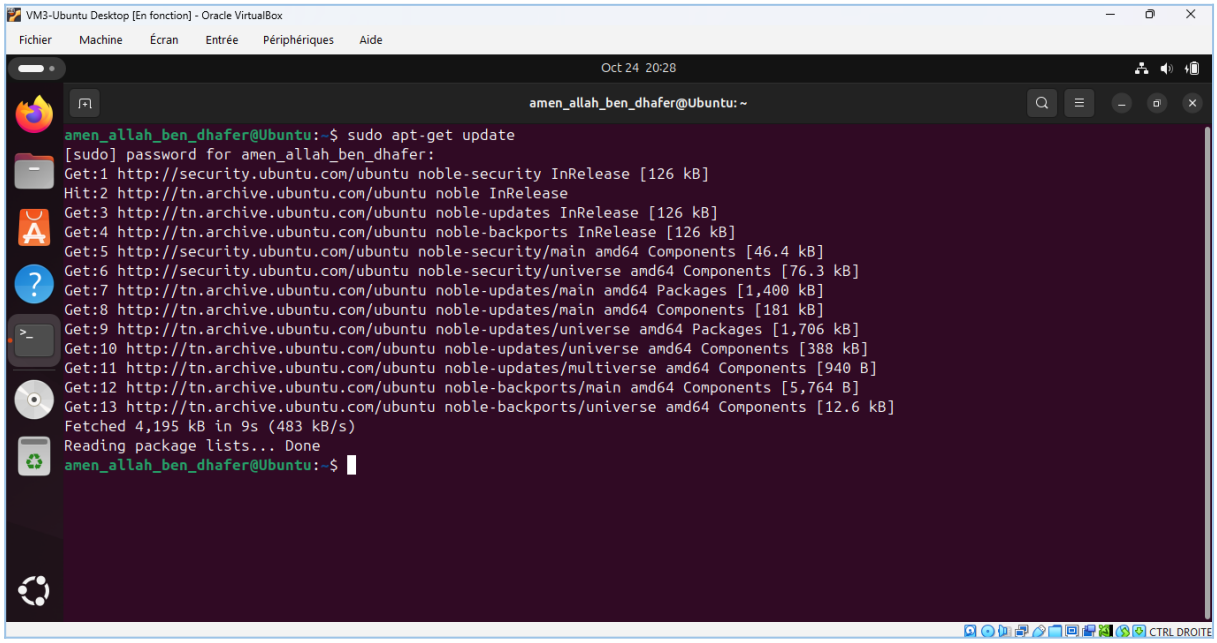


Figure 2 – *Mise à jour de la liste des paquets*

> **Observation :** Les dépôts Ubuntu **noble** (security, updates, backports) sont interrogés et la commande se termine par « Reading package lists… Done » : l'index est à jour et la VM accède bien à Internet.

Étape 2 : Installation de Docker

Le paquet `docker.io` est installé avec ses dépendances (containerd, runc, git…). Une confirmation est demandée avant le téléchargement.

```
$ sudo apt-get install docker.io
```

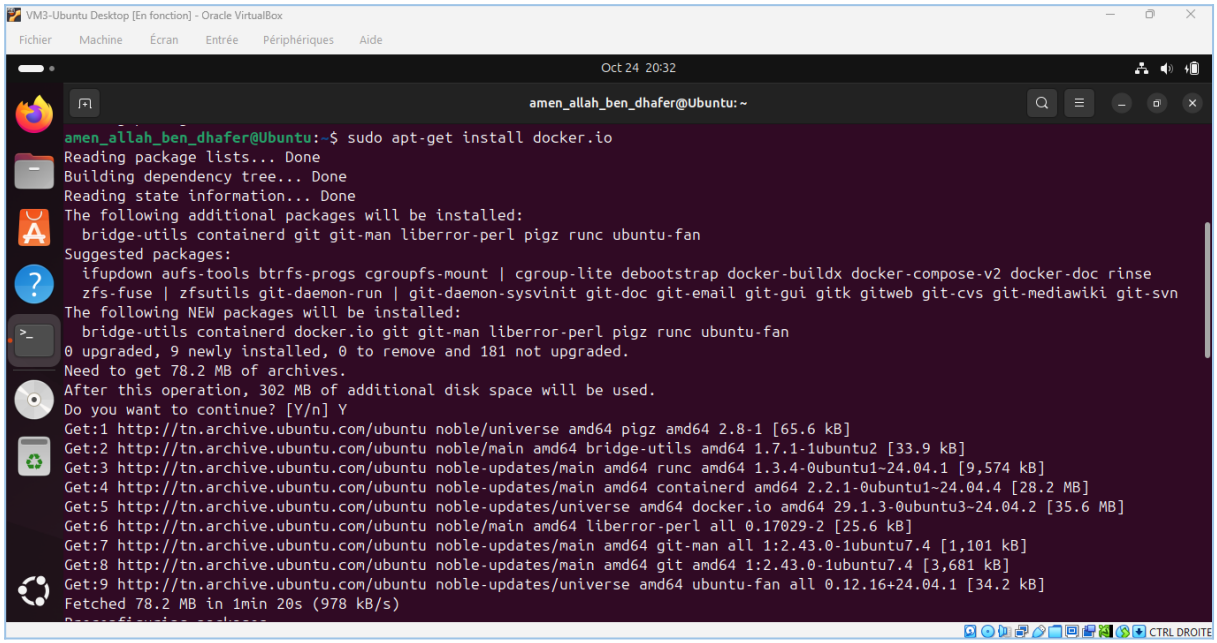


Figure 3 – *Lancement de l'installation : paquets à installer et confirmation*

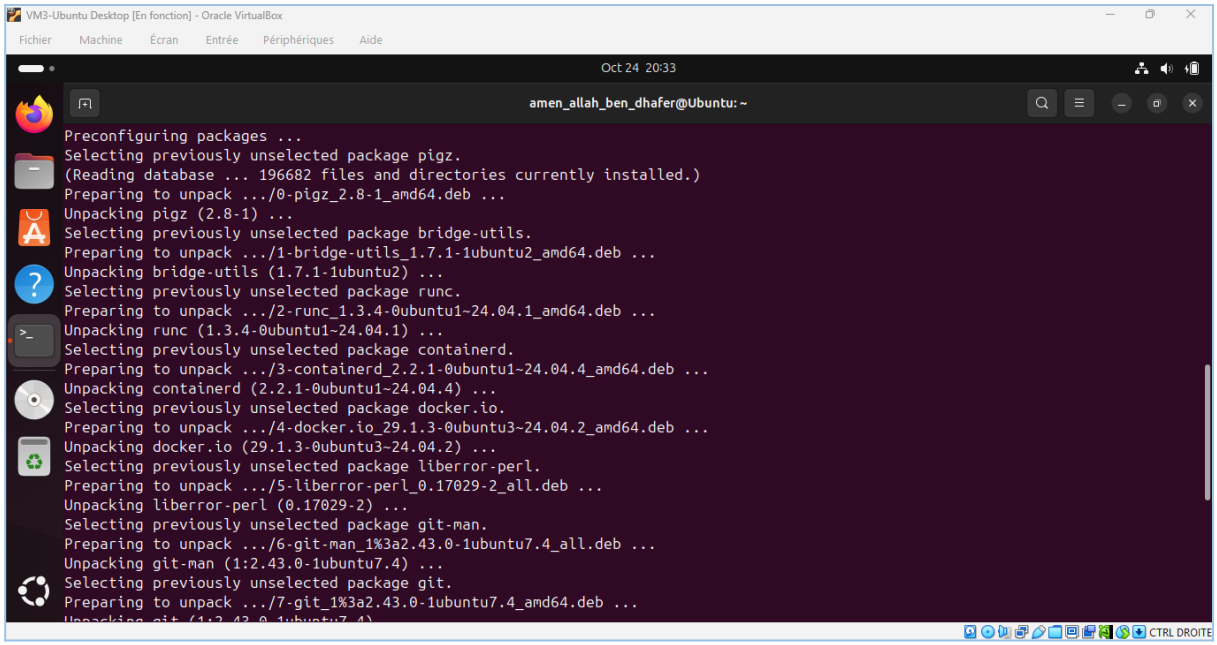


Figure 4 – *Dépaquetage des paquets téléchargés*

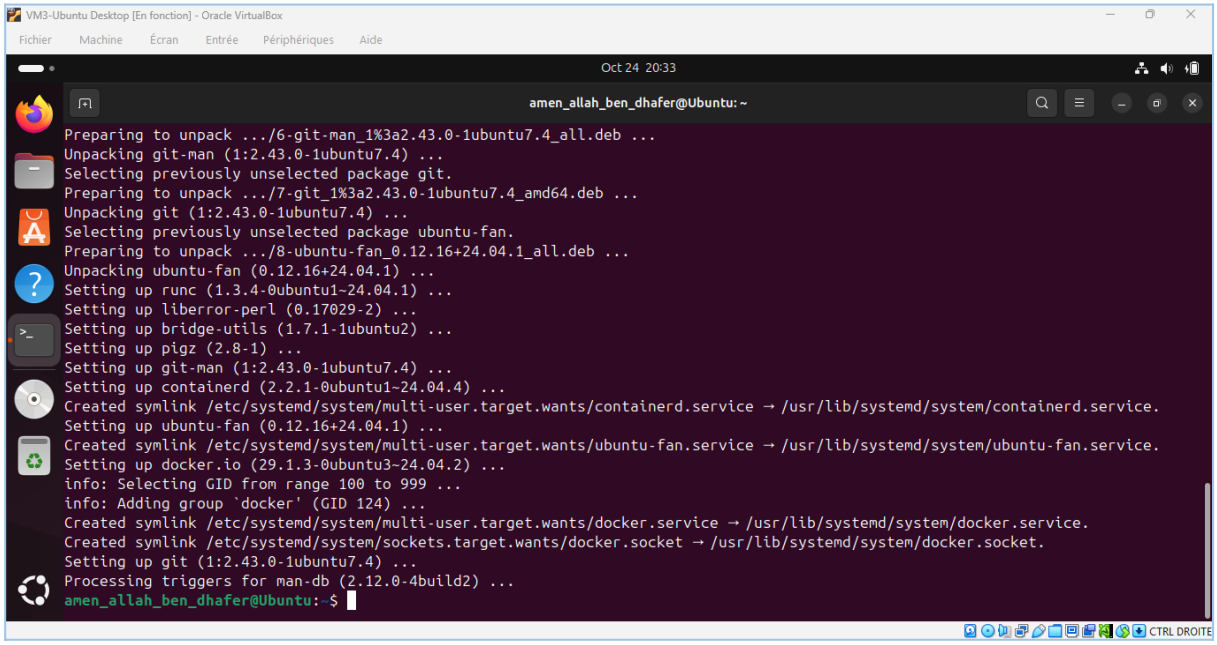


Figure 5 – *Configuration des paquets et fin de l'installation*

> **Observation :** L'installation ajoute **9 nouveaux paquets** (78,2 Mo à télécharger, environ 302 Mo d'espace disque). La version installée est **Docker 29.1.3**.
>
> Lors de la configuration, les services `docker.service` et `docker.socket` sont créés et le groupe système **docker** est ajouté automatiquement (GID 124).

1.3 Post-installation

1.3.1 Gestion du service Docker

Une fois installé, le service Docker est démarré, puis son état est vérifié. Il peut aussi être activé au démarrage du système.

```
$ sudo systemctl start docker
$ sudo systemctl status docker
```

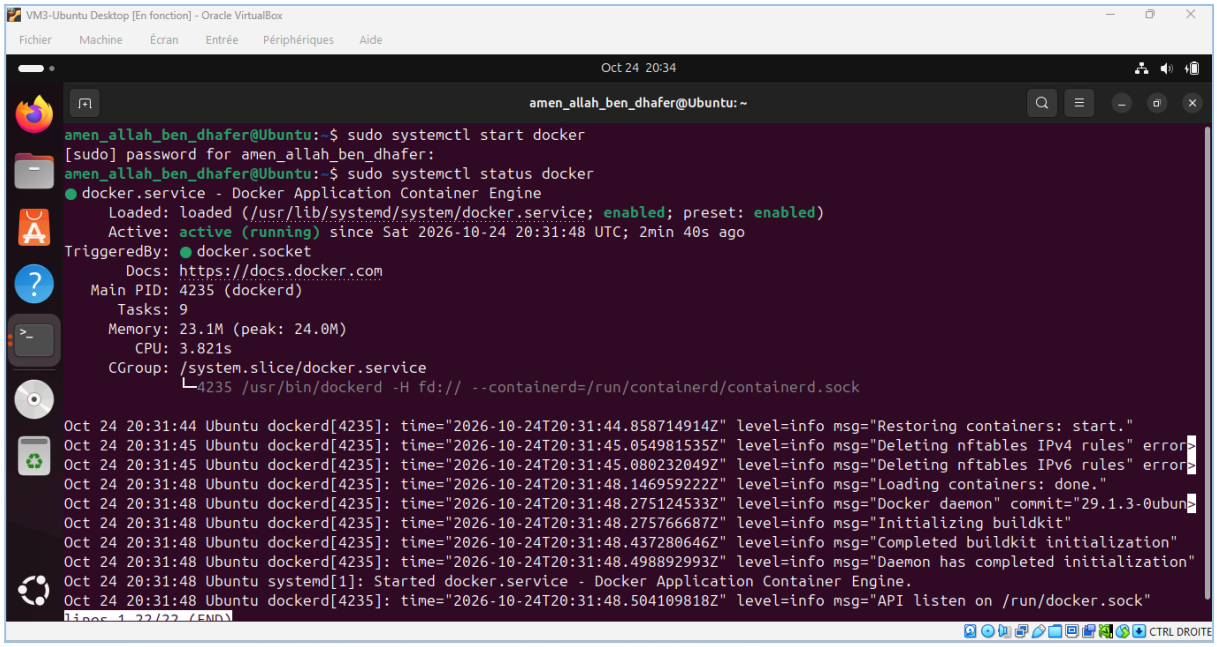


Figure 6 – *Démarrage du service Docker et vérification de son état*

Observation : Le service **docker.service** est « active (running) » et « enabled » : le démon `dockerd` (PID 4235) tourne et il est lancé via `docker.socket`.

```
$ sudo systemctl enable docker
```

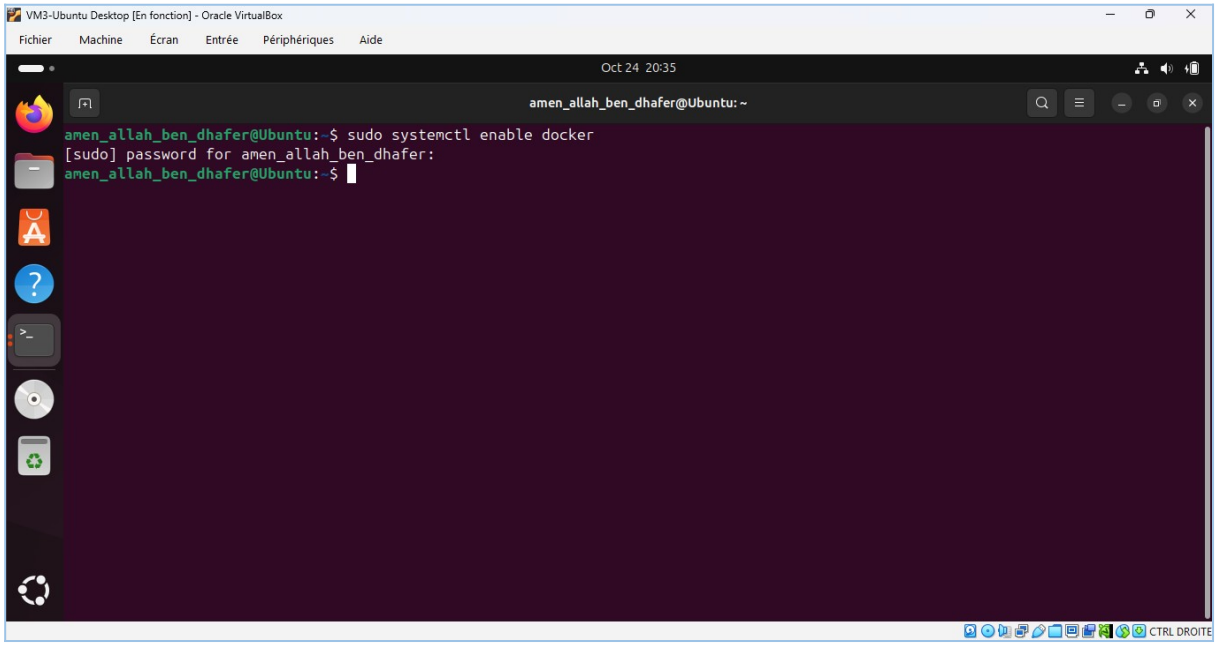


Figure 7 – *Activation du service au démarrage du système*

Observation : La commande ne retourne aucun message : le service était déjà configuré pour démarrer avec le système, comme l'indiquait l'état « enabled » de la figure 6.

1.3.2 Accord des privilèges (groupe docker)

Pour utiliser `docker` sans `sudo`, l'utilisateur doit appartenir au groupe **docker**, qui confère des privilèges équivalents à ceux de root. Le groupe est créé avec `groupadd` s'il n'existe pas, puis l'utilisateur y est ajouté avec `usermod`.

```
$ sudo groupadd docker
$ sudo usermod -aG docker $USER
```

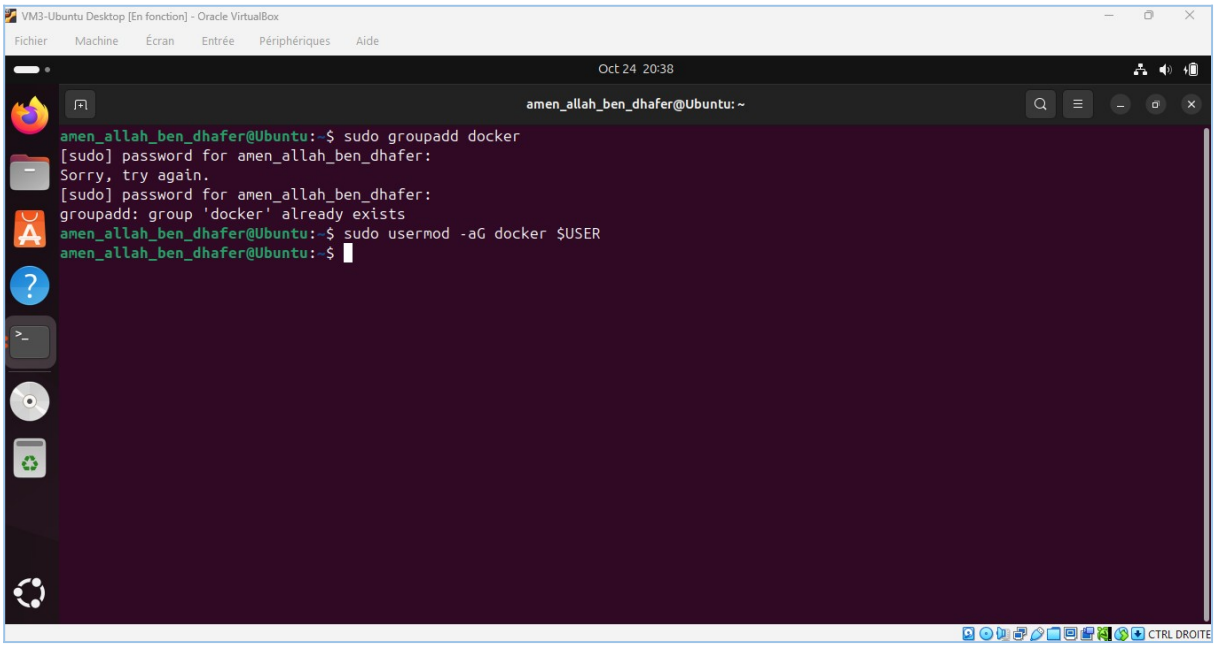


Figure 8 – *Création du groupe docker et ajout de l'utilisateur*

> **Observation :** `groupadd` indique que le groupe **docker existe déjà** : il a été créé lors de l'installation du paquet.
>
> L'option `-aG` de `usermod` ajoute l'utilisateur au groupe sans le retirer de ses autres groupes, et `$USER` contient le nom de l'utilisateur courant. La prise en compte du groupe demande une nouvelle session (voir 2.1.1).

2. Utilisation de la commande docker

La syntaxe générale de la commande est `docker [option] [command] [arguments]`.

Afficher les sous-commandes disponibles

```
$ docker
```

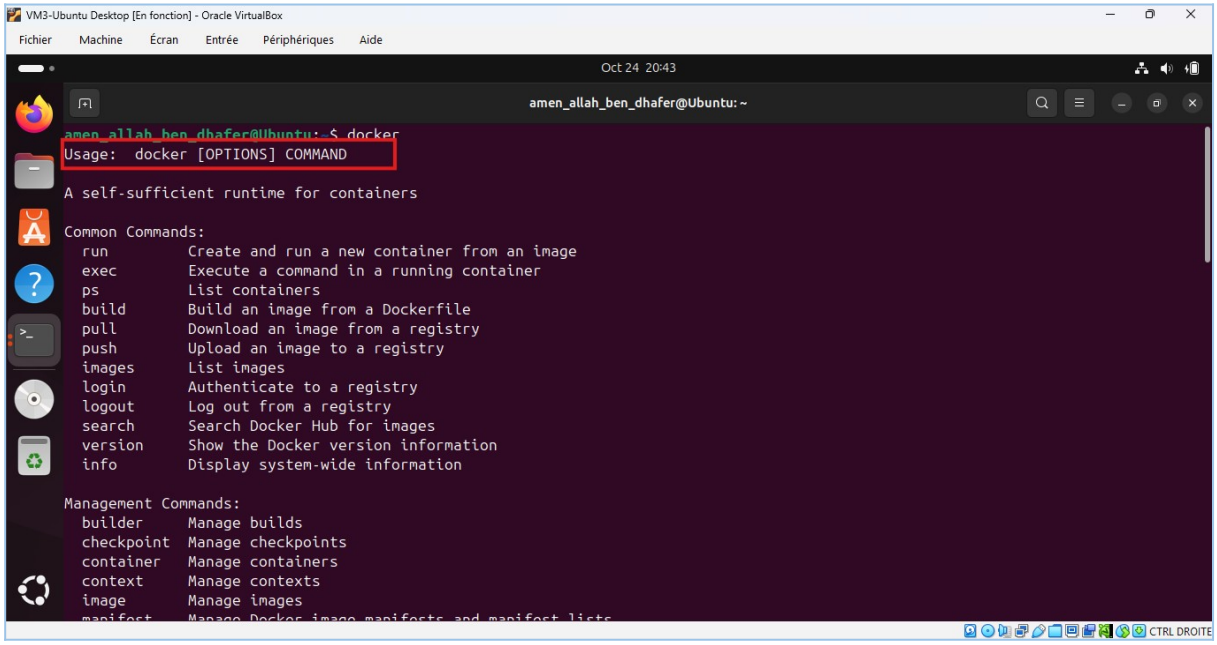


Figure 9 – *Résultat de la commande docker : syntaxe et commandes courantes*

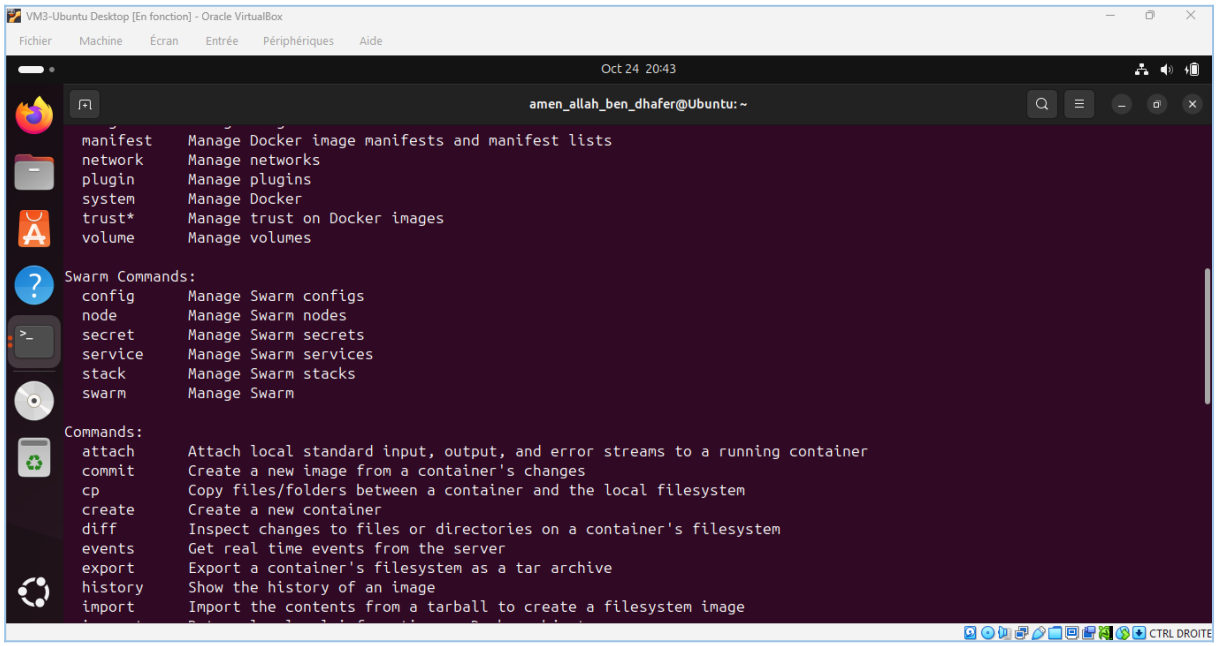


Figure 10 – *Suite de l'aide : commandes de gestion, commandes Swarm et autres commandes*

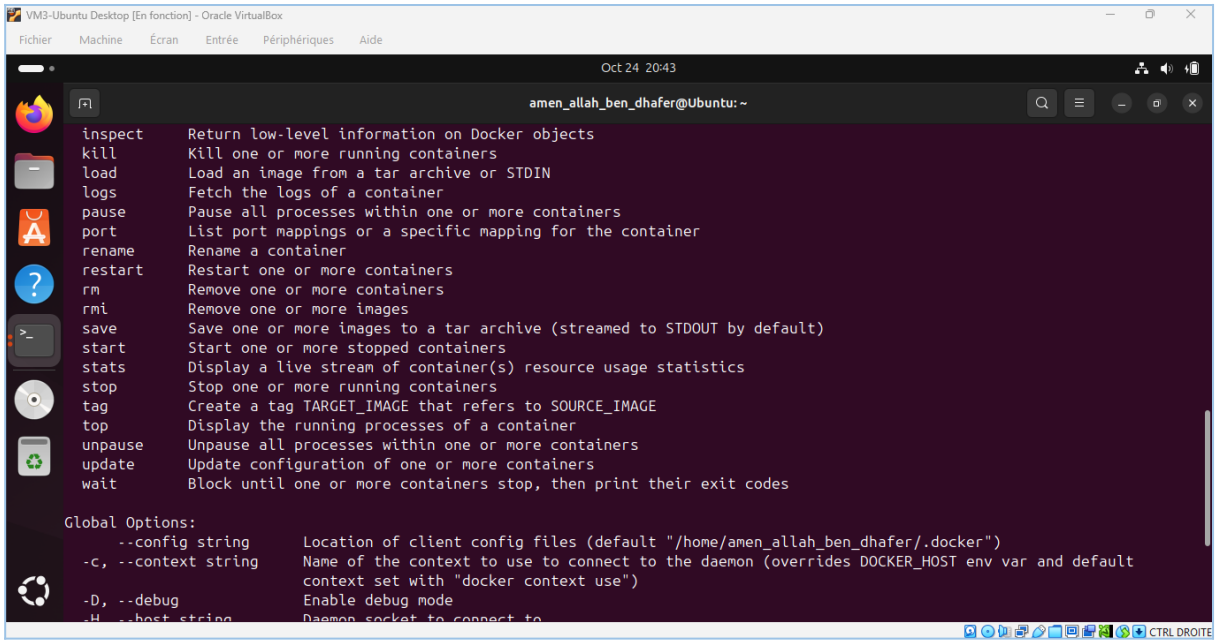


Figure 11 – *Suite de la liste des commandes et début des options globales*

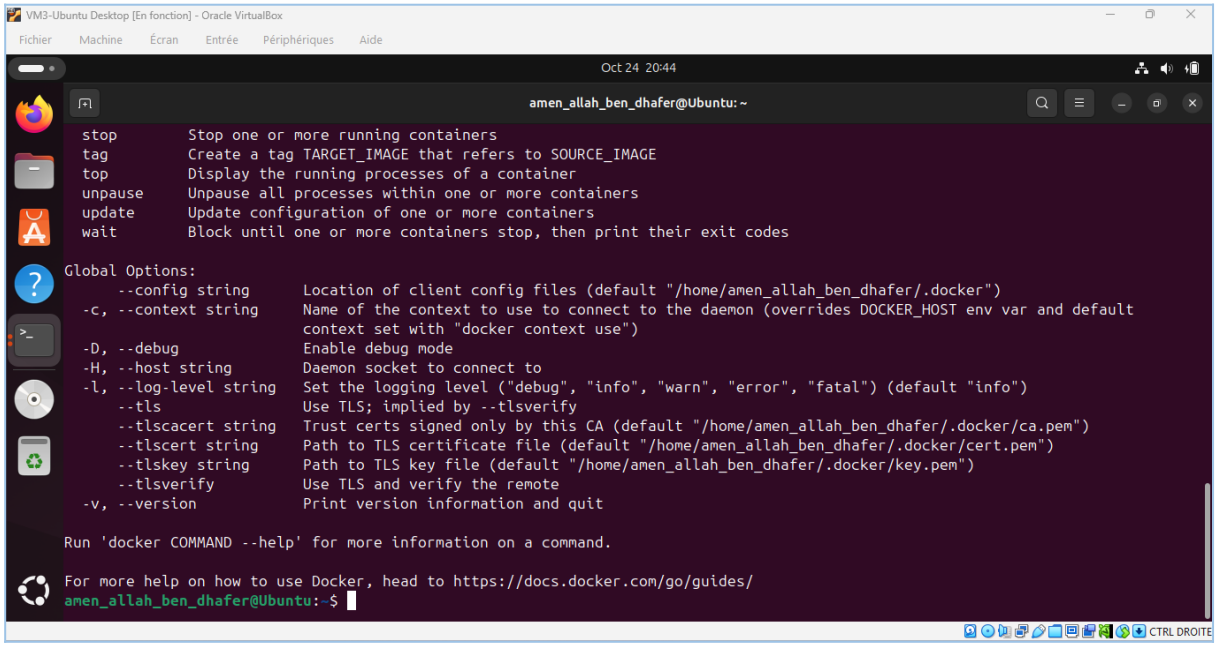


Figure 12 – *Fin de l'aide : options globales*

> **Observation :** L'aide classe les commandes en familles : **commandes courantes** (`run`, `exec`, `ps`, `build`, `pull`, `push`, `images`, `search`...), **commandes de gestion** (`container`, `image`, `network`, `volume`, `system`...), **commandes Swarm** et commandes d'administration (`create`, `start`, `stop`, `rm`, `rmi`...). Elle se termine par les options globales.

Afficher les options d'une commande (exemple : run)

```
$ docker run --help
```

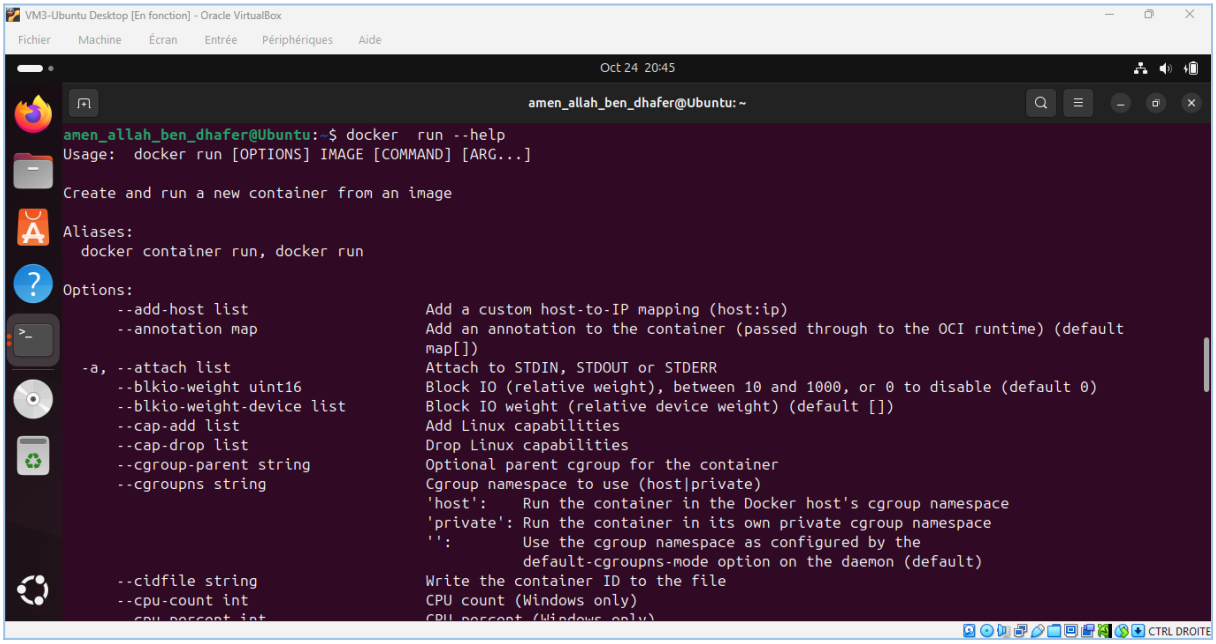


Figure 13 – *Début de l'aide de docker run : syntaxe et premières options*

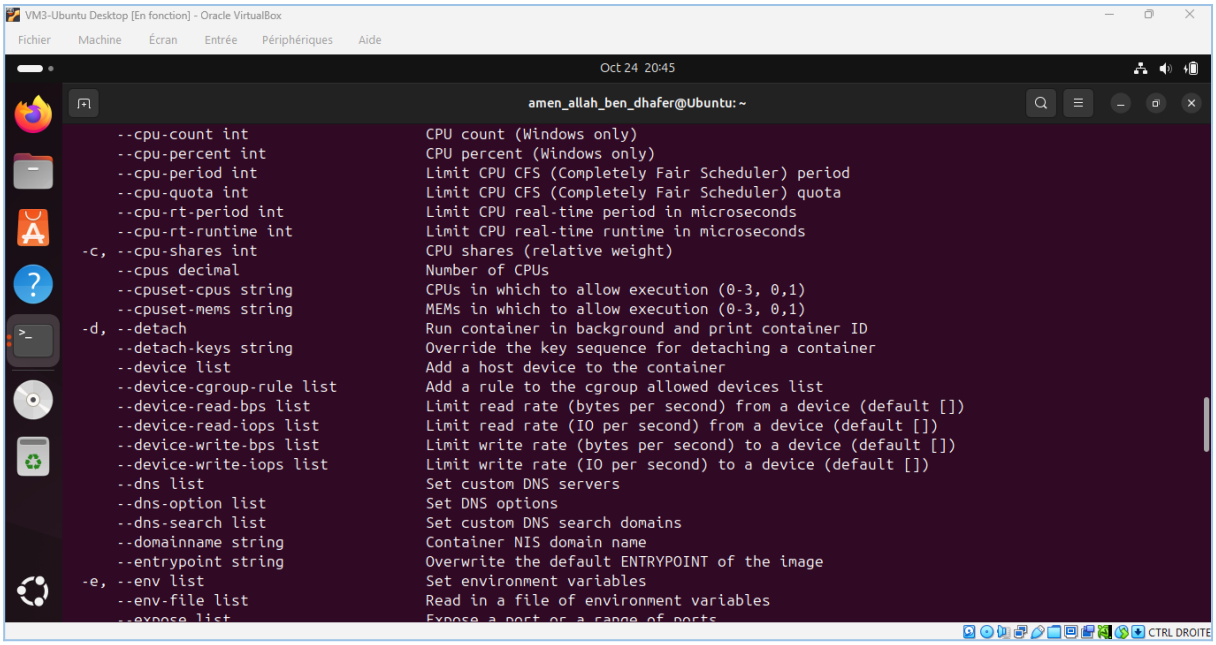


Figure 14 – *Options de docker run (suite) : options CPU, -d, -e*

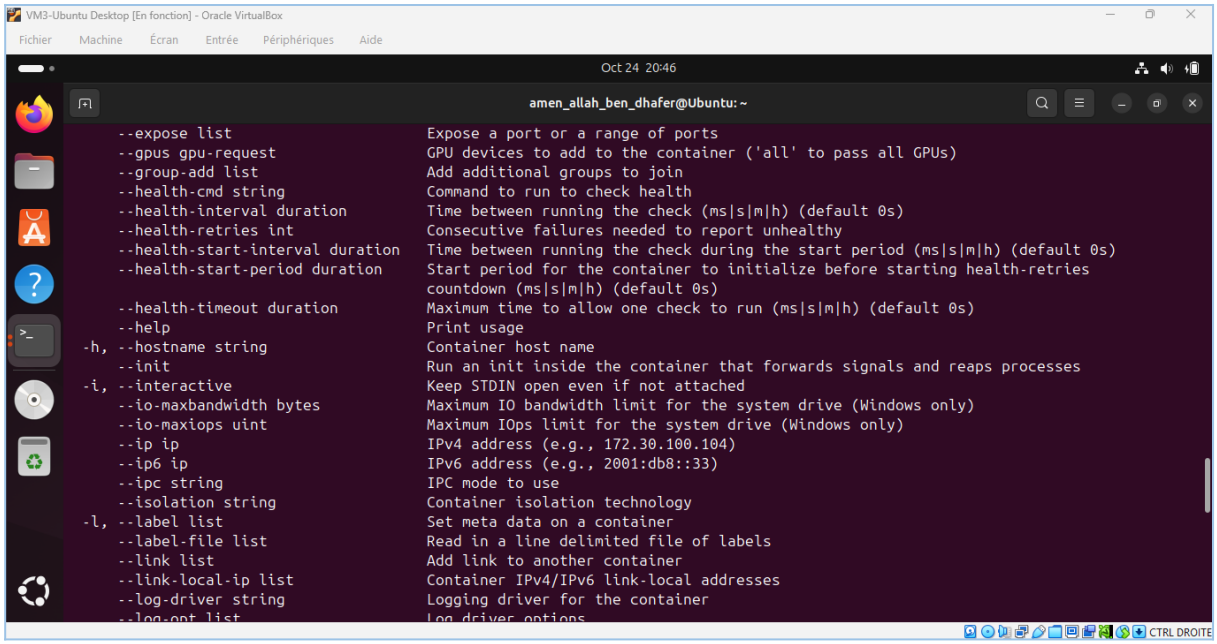


Figure 15 – *Options de docker run (suite) : --expose, -i, -l*

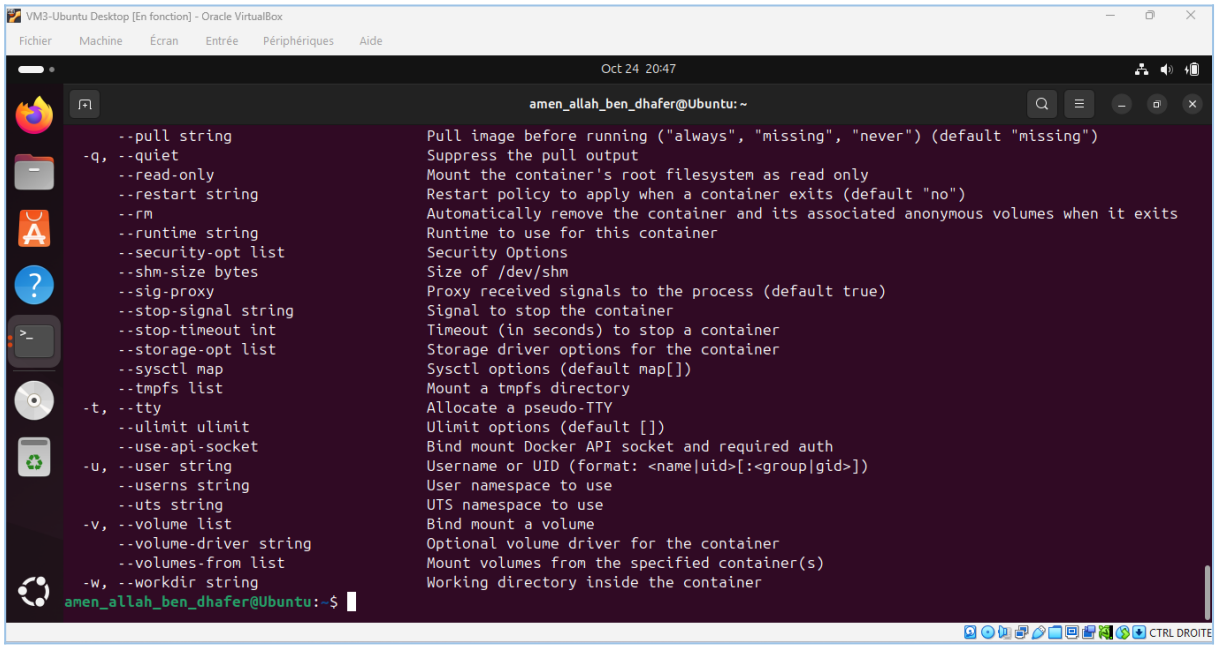


Figure 16 – *Fin de l'aide : --rm, -t, -u, -v, -w*

> **Observation :** L'aide donne la syntaxe `docker run [OPTIONS] IMAGE [COMMAND] [ARG...]`. Les options utiles dans ce TP sont : `-d` (exécution en arrière-plan), `-i` et `-t` (mode interactif avec terminal), `--rm` (suppression automatique du conteneur à son arrêt), `-e` (variables d'environnement), `-v` (volumes) et `-w` (répertoire de travail).

2.1 Gestion des images

2.1.1 Lister les images

La commande `docker images` affiche les images disponibles en local.

```
$ docker images
```

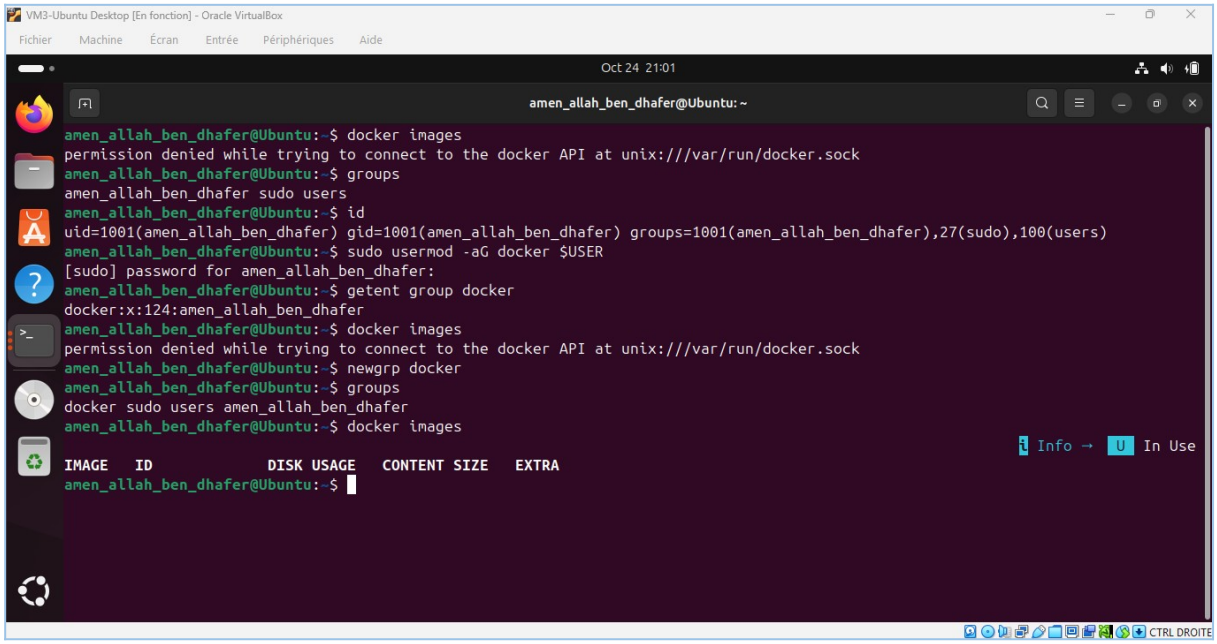


Figure 17 – *docker images : erreur de permission puis résolution avec newgrp docker*

> **Observation :** La première tentative échoue avec **permission denied** : l'utilisateur vient d'être ajouté au groupe `docker`, mais la session ouverte ne l'a pas encore pris en compte (`groups` ne l'affiche pas). Après vérification avec `getent group docker`, la commande `newgrp docker` active le groupe dans le terminal ; `groups` affiche alors `docker` et `docker images` fonctionne.
>
> La liste est **vide** : aucune image n'a encore été téléchargée.

Pour obtenir une première image locale, on exécute `docker run hello-world` : l'image est téléchargée depuis Docker Hub puis exécutée.

```
$ docker run hello-world
$ docker images
```

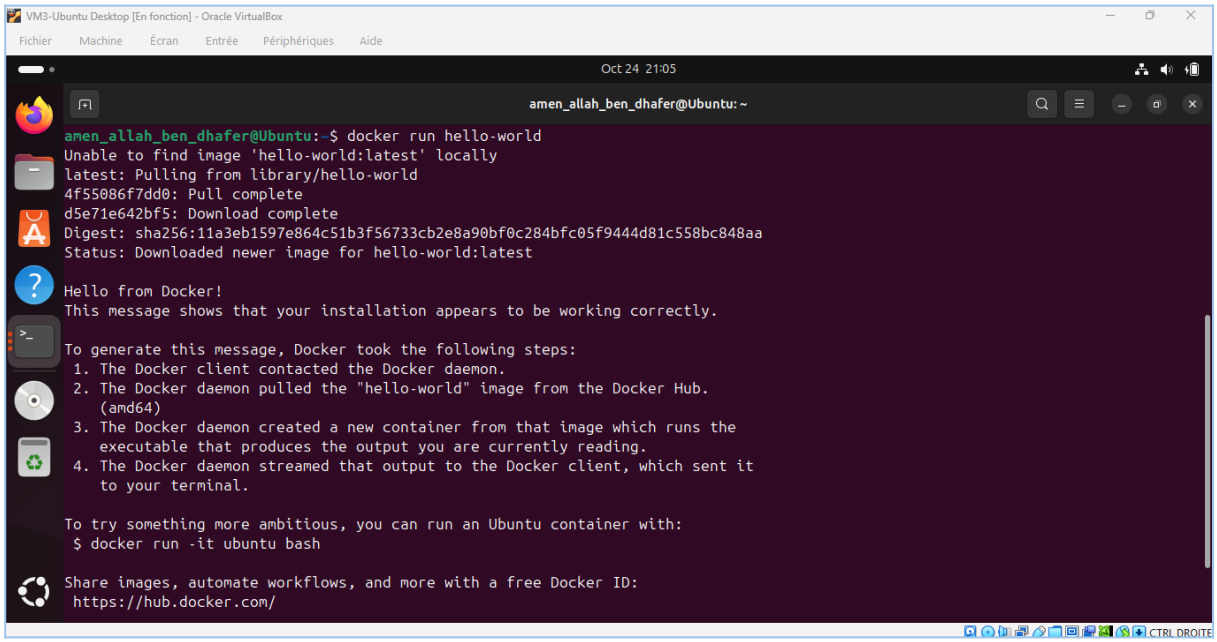


Figure 18 – *Exécution de docker run hello-world : téléchargement de l'image et message d'accueil*

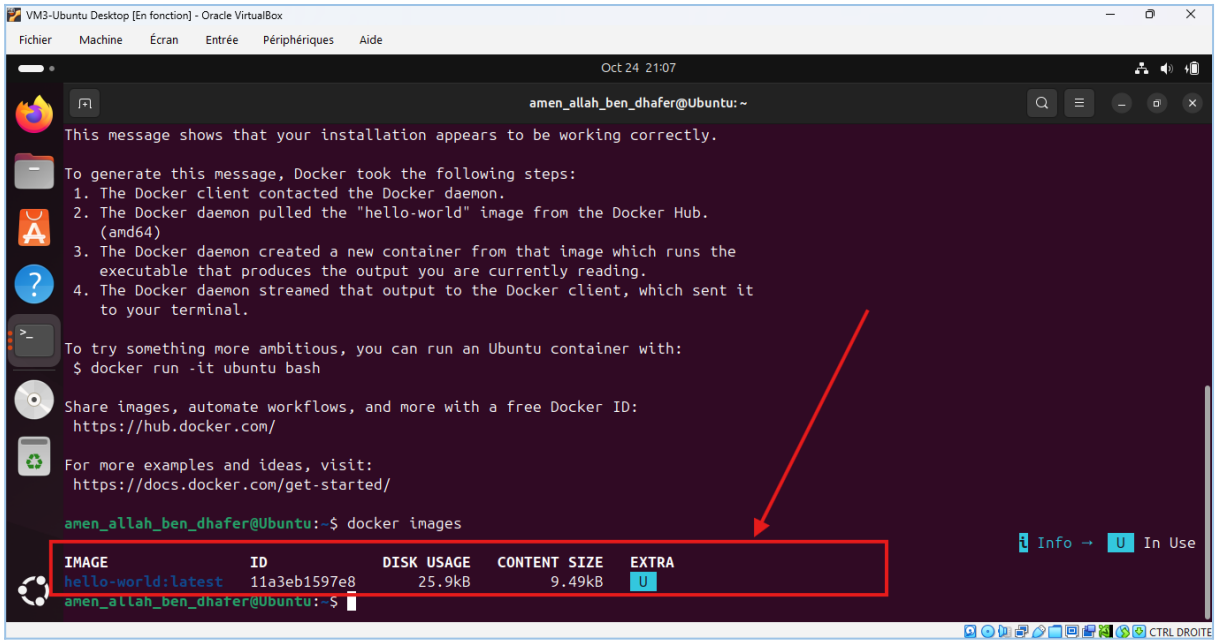


Figure 19 – *Liste des images après le téléchargement de hello-world*

> **Observation :** Docker ne trouve pas l'image en local (« Unable to find image 'hello-world:latest' locally »), la télécharge, puis affiche « Hello from Docker! » : l'installation fonctionne.
>
> `docker images` affiche maintenant l'image **hello-world:latest** avec les colonnes **IMAGE** (nom et tag), **ID** (identifiant unique, ici `11a3eb1597e8`), **DISK USAGE** (espace occupé : 25,9 kB) et **CONTENT SIZE** (taille du contenu : 9,49 kB). Les anciennes versions de Docker affichaient REPOSITORY, TAG, IMAGE ID, CREATED et SIZE.

2.1.2 Chercher une image

La commande `docker search` interroge le registre Docker Hub. La recherche porte sur le nom de l'image, l'utilisateur ou la description.

```
$ docker search centos
```

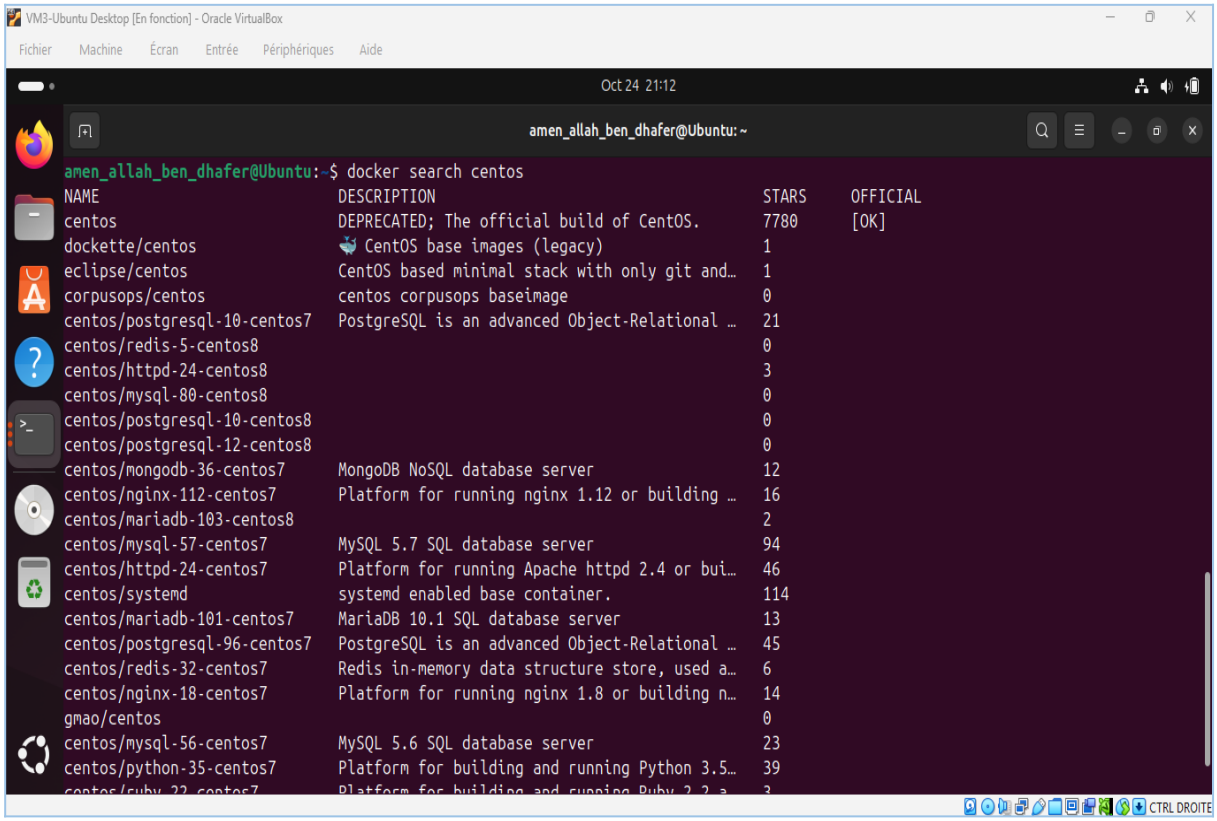


Figure 20 – *Résultat de docker search centos (début de la liste)*

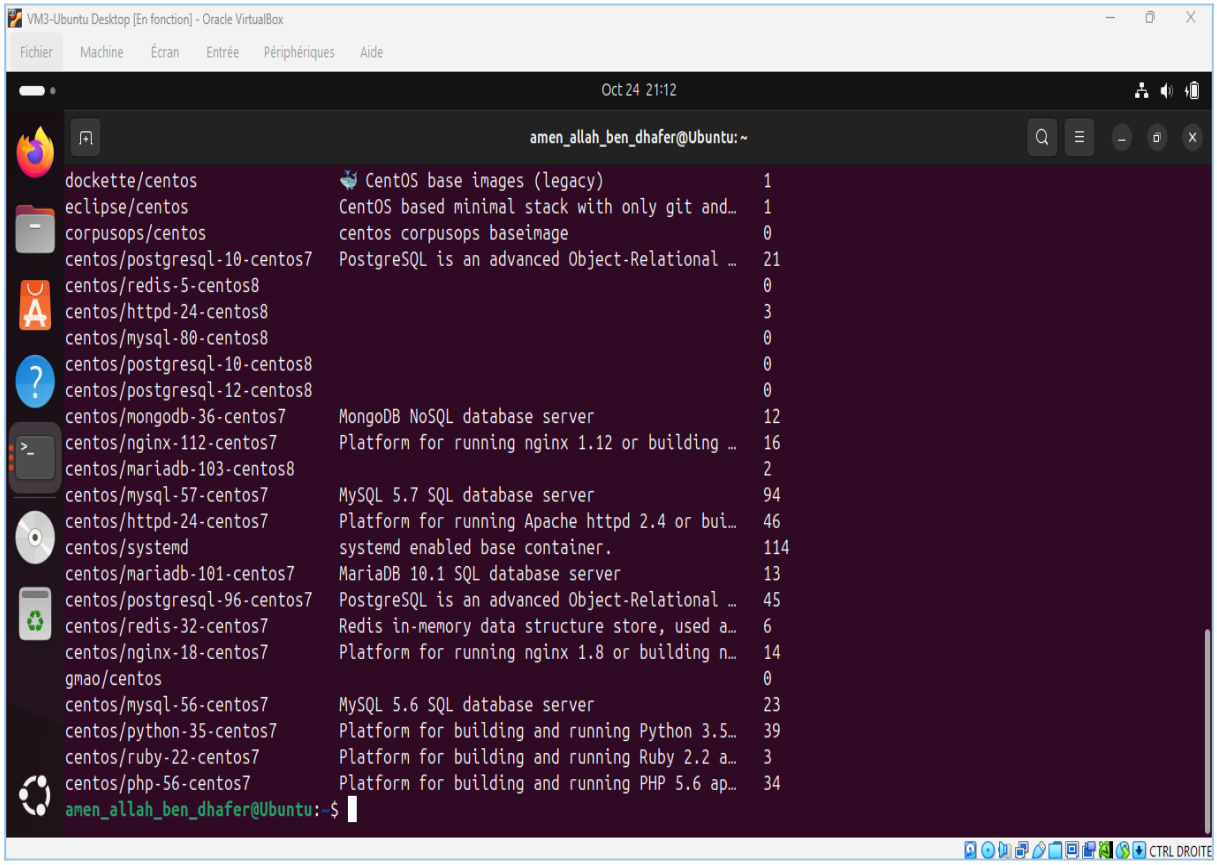


Figure 21 – *Suite de la liste des résultats*

Observation : L'image officielle **centos** arrive en premier (7780 étoiles, marquée **[OK]** dans la colonne OFFICIAL) avec la mention « DEPRECATED ». Les autres résultats sont des images d'utilisateurs, dont le nom est préfixé par celui du dépôt (`centos/systemd`, `eclipse/centos`…).

Les colonnes affichées sont NAME, DESCRIPTION, STARS (notation) et OFFICIAL (image officielle).

Filtrer par nombre d'étoiles

```
$ docker search --filter=stars=5 centos
```

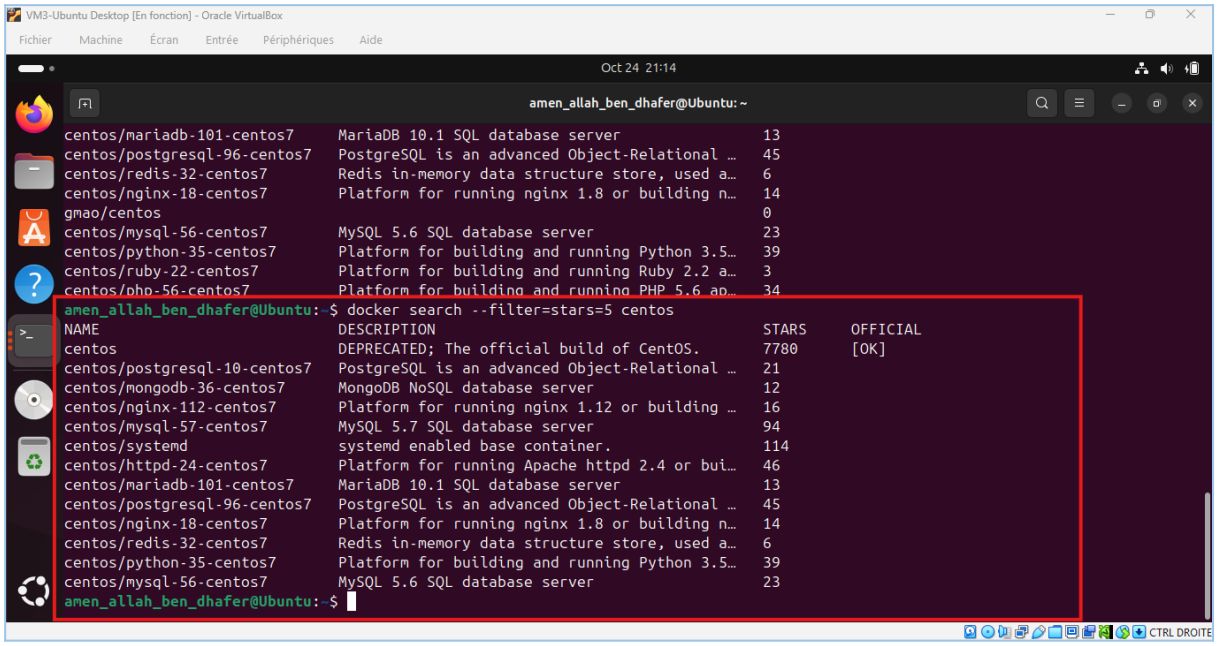


Figure 22 – *Recherche filtrée : images centos avec au moins 5 étoiles*

Observation : L'option `--filter=stars=5` ne garde que les images notées au moins 5 étoiles : les résultats à 0, 1 ou 3 étoiles (par exemple `eclipse/centos` ou `corpusops/centos`) disparaissent de la liste.

2.1.3 Télécharger une image

La commande `docker pull` télécharge une image depuis le registre. Essayons avec l'image `centos`.

```
$ docker pull centos
```

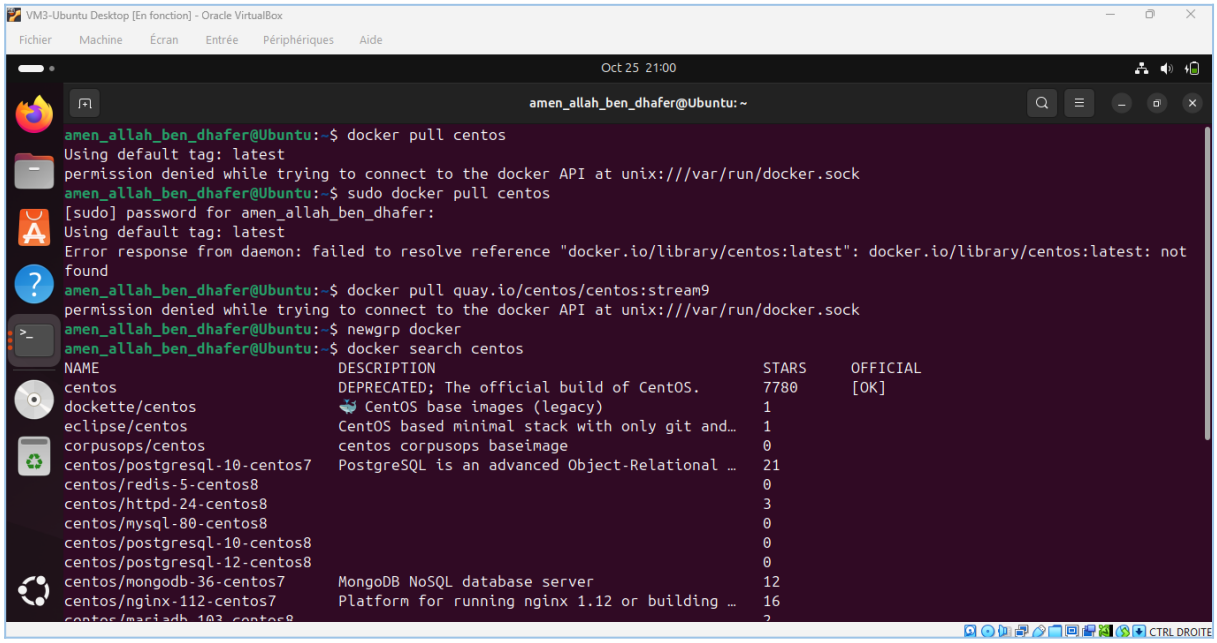


Figure 23 – *Tentatives de docker pull centos et message d'erreur*

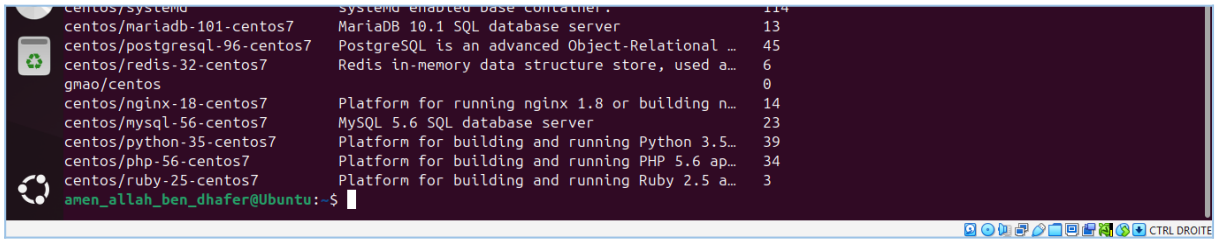


Figure 24 – *Suite de la liste obtenue avec docker search centos*

Réponses aux questions du TP : Quel est le message d'erreur obtenu ? « Error response from daemon: failed to resolve reference "docker.io/library/centos:latest": docker.io/library/centos:latest: not found ». Sans `sudo`, la commande est d'abord refusée (permission denied) car la session n'avait pas encore le groupe docker ; avec `sudo`, elle atteint le registre et retourne cette erreur.

Pourquoi cette erreur ? Sans tag précisé, Docker cherche le tag `latest`. Or l'image officielle `centos` est dépréciée (mention « DEPRECATED » dans la recherche) : CentOS Linux n'est plus maintenu et le tag `latest` n'existe plus sur Docker Hub.

Quelle version utiliser ? Une version avec un tag explicite, par exemple `centos:centos7`, téléchargée avec succès à l'étape 2.2.1 (ou `centos:centos8`, utilisée en 2.2.7).

2.1.4 Supprimer une image

La commande `docker rmi` supprime une image locale désignée par son identifiant ou son nom. Elle ne supprime pas une image utilisée par un conteneur.

```
$ docker rmi <image ID_or_Name>
$ docker rmi 11a3eb1597e8
```

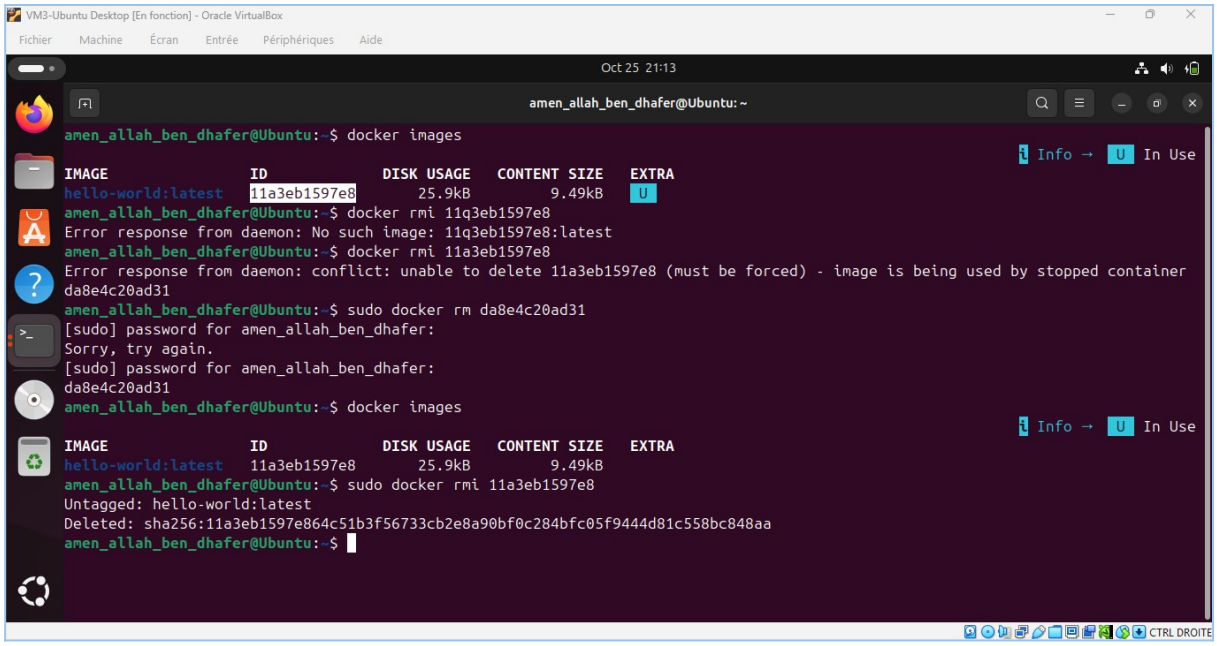


Figure 25 – *Suppression de l'image hello-world : erreurs successives puis suppression réussie*

Observation : Identifiant erroné : `docker rmi 11q3eb1597e8` retourne « No such image » (faute de frappe dans l'identifiant).

Image utilisée : avec le bon identifiant, l'erreur « conflict: unable to delete 11a3eb1597e8 (must be forced) - image is being used by stopped container da8e4c20ad31 » indique qu'un conteneur arrêté, créé à partir de l'image, la retient.

Résolution : le conteneur est supprimé avec `sudo docker rm da8e4c20ad31`, puis `sudo docker rmi 11a3eb1597e8` affiche « Untagged » et « Deleted » : l'image est supprimée.

Supprimer toutes les images

```
$ docker rmi $(docker images -a -q)
```

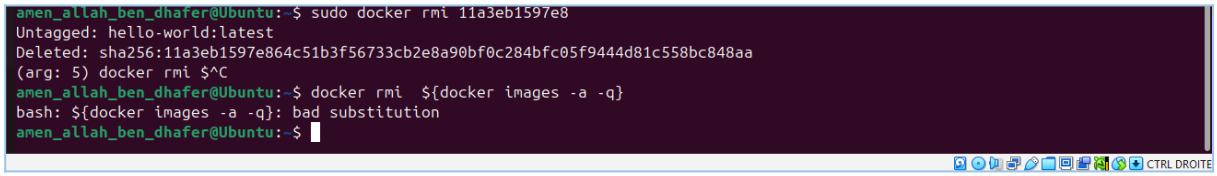


Figure 26 – *Tentative de suppression de toutes les images : erreur « bad substitution »*

Observation : La commande a été saisie avec des accolades `${...}` : bash retourne « bad substitution », car la substitution de commande s'écrit avec des **parenthèses** `$(...)`.

La syntaxe correcte est `docker rmi $(docker images -a -q)` : `docker images -a -q` renvoie uniquement les identifiants de toutes les images, que `docker rmi` supprime ensuite.

2.2 Gestion des conteneurs

2.2.1 Créer un conteneur à partir d'une image

La commande `docker create` crée un conteneur à partir d'une image **sans le démarrer**. L'option `--name` lui attribue un nom.

```
$ docker create --name MyFirstContainer centos:centos7
```

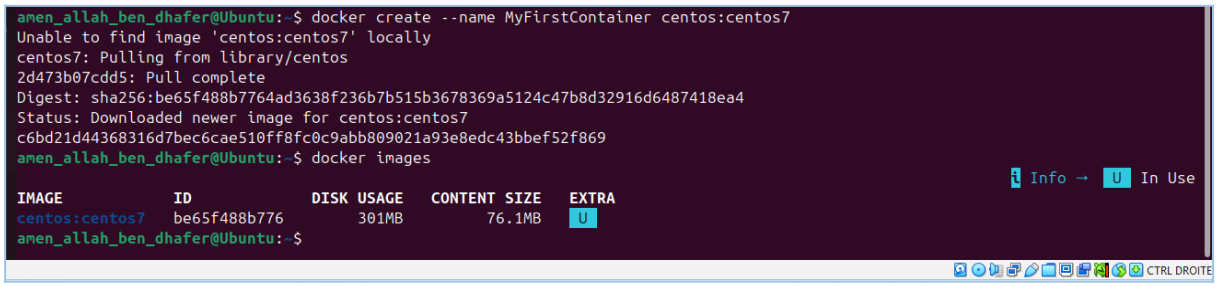


Figure 27 – *Création du conteneur MyFirstContainer à partir de centos:centos7*

> **Observation :** L'image `centos:centos7` n'existant pas en local, Docker la télécharge automatiquement (« Pull complete »), crée le conteneur et affiche son identifiant. `docker images` montre alors l'image **centos:centos7** (ID `be65f488b776`, 301 Mo).

2.2.2 Lister les conteneurs en cours (actifs)

```
$ docker ps
```

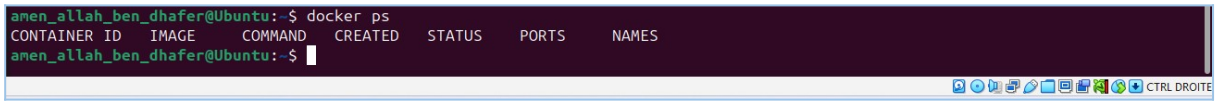


Figure 28 – *docker ps : aucun conteneur actif*

> **Observation :** Seuls les conteneurs en cours d'exécution sont listés. La liste est vide : le conteneur créé n'a pas encore été démarré.

2.2.3 Lister tous les conteneurs

```
$ docker ps -a
```

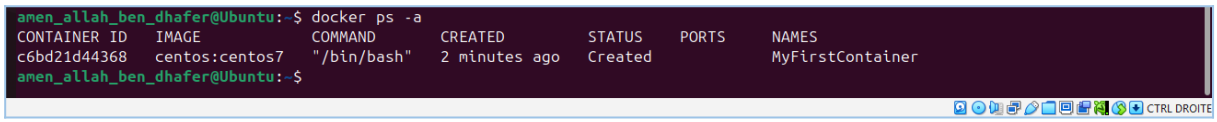


Figure 29 – *docker ps -a : liste de tous les conteneurs*

> **Observation : CONTAINER ID** : identifiant unique du conteneur (`c6bd21d44368`) ; **IMAGE** : image utilisée (`centos:centos7`) ; **COMMAND** : commande lancée par défaut (`/bin/bash`).
>
> **CREATED** : date de création ; **STATUS** : état du conteneur, ici *Created* (créé mais non démarré) ; **PORTS** : ports exposés (aucun) ; **NAMES** : nom attribué (`MyFirstContainer`).

2.2.4 Démarrer un conteneur

```
$ docker start <container-id>
$ docker start c6bd21d44368
```

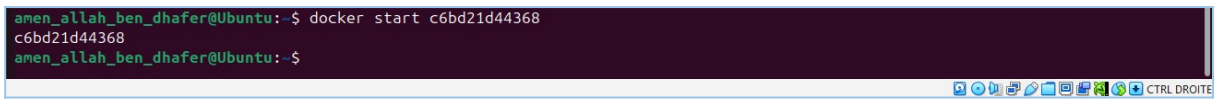


Figure 30 – *Démarrage du conteneur à partir de son identifiant*

> **Observation :** La commande affiche l'identifiant du conteneur démarré. Le conteneur peut aussi être désigné par son nom (`docker start MyFirstContainer`).

2.2.5 Arrêter un conteneur

```
$ docker stop c6bd21d44368
```



Figure 31 – *Arrêt du conteneur*

> **Observation :** `docker stop` arrête le conteneur et affiche son identifiant. Comme pour `start`, le nom `MyFirstContainer` peut remplacer l'identifiant.

2.2.6 Créer et démarrer un conteneur

```
$ docker run centos:centos7
```

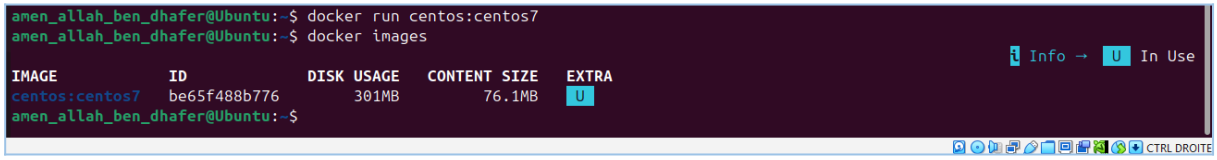


Figure 32 – *Création et démarrage d'un conteneur avec docker run*

> **Observation :** `docker run` regroupe `docker create` et `docker start` : l'image étant déjà présente (`docker images`), aucun téléchargement n'est nécessaire.
>
> Le conteneur lance `/bin/bash`, qui se termine immédiatement faute de terminal attaché : il passe à l'état **Exited (0)** (voir figure 33).

2.2.7 Supprimer un conteneur

La commande `docker rm` supprime un ou plusieurs conteneurs désignés par leur nom ou leur identifiant. Par défaut, elle refuse de supprimer un conteneur en cours d'exécution : l'option `-f` force la suppression.

Supprimer un conteneur

```
$ docker rm c6bd21d44368
$ docker ps -q
$ docker ps -a
```

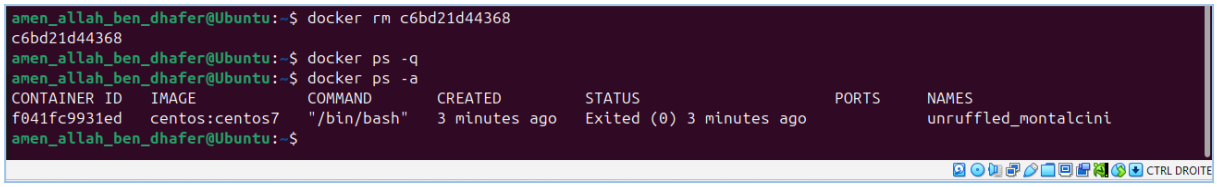


Figure 33 – *Suppression du conteneur MyFirstContainer et vérification*

> **Observation :** `MyFirstContainer` est supprimé. `docker ps -q` (identifiants des conteneurs actifs) ne renvoie rien, et `docker ps -a` ne montre plus que le conteneur issu de `docker run`, au statut **Exited (0)**.

Supprimer plusieurs conteneurs, quel que soit leur état

```
$ docker rm -f <container ID_or_Name> <container ID_or_Name>
```

```
amen_allah_ben_dhafer@Ubuntu:~$ docker run centos:centos8
Unable to find image 'centos:centos8' locally
centos8: Pulling from library/centos
a1d0c7532777: Pull complete
Digest: sha256:a27fd8080b517143cbbbab9dfb7c8571c40d67d534bbdee55bd6c473f432b177
Status: Downloaded newer image for centos:centos8
amen_allah_ben_dhafer@Ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS                      PORTS     NAMES
80b514149ff1   centos:centos8  "/bin/bash"   18 seconds ago   Exited (0) 3 seconds ago              festive_elgamal
f041fc9931ed   centos:centos7  "/bin/bash"   6 minutes ago    Exited (0) 6 minutes ago              unruffled_montalcini
amen_allah_ben_dhafer@Ubuntu:~$ docker rm -f 80b514149ff1 f041fc9931ed
80b514149ff1
f041fc9931ed
amen_allah_ben_dhafer@Ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
amen_allah_ben_dhafer@Ubuntu:~$
```

Figure 34 – *Création d'un conteneur centos8 puis suppression forcée de deux conteneurs*

Observation : `docker run centos:centos8` télécharge l'image et crée un second conteneur. `docker ps -a` en liste deux, tous deux **Exited (0)**.

`docker rm -f 80b514149ff1 f041fc9931ed` supprime les deux en une seule commande, et la liste devient vide.

Supprimer un conteneur à sa fermeture, puis supprimer tous les conteneurs

```
$ docker run --rm centos:centos7
$ docker rm $(docker ps -a -q)
```

```
amen_allah_ben_dhafer@Ubuntu:~$ docker run --rm centos:centos7
amen_allah_ben_dhafer@Ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE           COMMAND       CREATED             STATUS    PORTS     NAMES
995fa5ff2359   centos:centos7  "/bin/bash"   About a minute ago  Created             MyFirstContainer
amen_allah_ben_dhafer@Ubuntu:~$ docker rm $(docker ps -a -q)
995fa5ff2359
amen_allah_ben_dhafer@Ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
amen_allah_ben_dhafer@Ubuntu:~$
```

Figure 35 – *Option --rm puis suppression de tous les conteneurs*

Observation : Avec `--rm`, le conteneur est **supprimé dès qu'il s'arrête** : `docker ps -a` ne liste que `MyFirstContainer` (statut *Created*).

`docker rm $(docker ps -a -q)` supprime ensuite tous les conteneurs (`-q` ne renvoie que les identifiants) et la liste devient vide.

Supprimer tous les conteneurs, quel que soit leur état

```
$ docker rm -f $(docker ps -a -q)
```

```
amen_allah_ben_dhafer@Ubuntu:~$ docker create --name MyFirstContainer centos:centos7
af5a999c300a442fd2e640294fd980f5608f7cce8436b3c68e501784bc52d0b7
amen_allah_ben_dhafer@Ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE           COMMAND       CREATED         STATUS    PORTS     NAMES
af5a999c300a   centos:centos7  "/bin/bash"   5 seconds ago   Created             MyFirstContainer
amen_allah_ben_dhafer@Ubuntu:~$ docker rm -f $(docker ps -a -q)
af5a999c300a
amen_allah_ben_dhafer@Ubuntu:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
amen_allah_ben_dhafer@Ubuntu:~$
```

Figure 36 – *Suppression forcée de tous les conteneurs*

Observation : Le conteneur `MyFirstContainer` est recréé, puis `docker rm -f $(docker ps -a -q)` supprime tous les conteneurs, y compris ceux qui sont en cours d'exécution. `docker ps -a` confirme que la liste est vide.

2.2.8 Exécuter une commande dans un conteneur

Une commande peut être passée au démarrage du conteneur, en mode interactif (`-i -t`) ou avec `docker exec` sur un conteneur actif. Ici, la commande `uname -a` est passée à des conteneurs `ubuntu` pour afficher les informations système, puis comparée à celles de la machine hôte.

Premier conteneur

```
$ docker run ubuntu uname -a
```

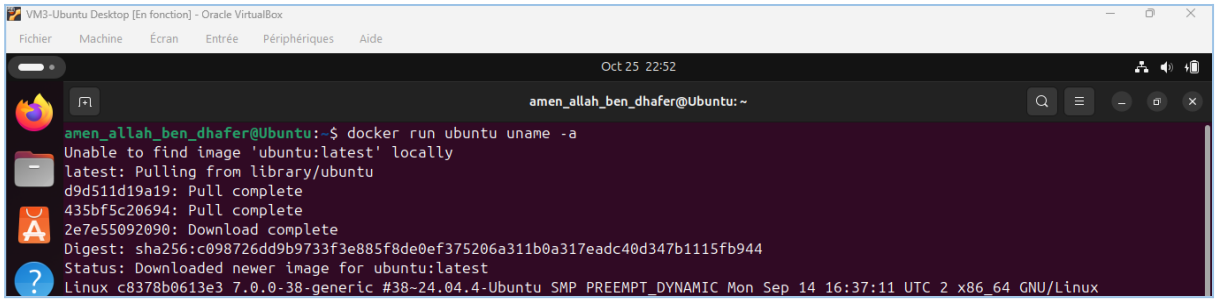


Figure 37 – *Premier conteneur : docker run ubuntu uname -a*

> **Observation :** L'image `ubuntu` est téléchargée automatiquement, puis le conteneur affiche les informations du noyau : **7.0.0-38-generic**, architecture x86_64.

Deuxième conteneur

```
$ docker run ubuntu uname -a
```

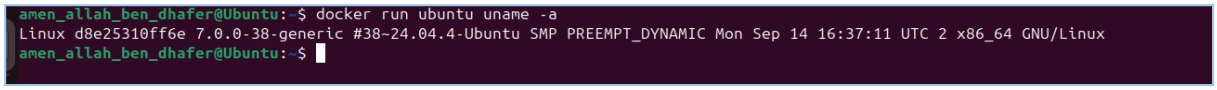


Figure 38 – *Deuxième conteneur : docker run ubuntu uname -a*

Dans le terminal de la machine

```
$ uname -a
```

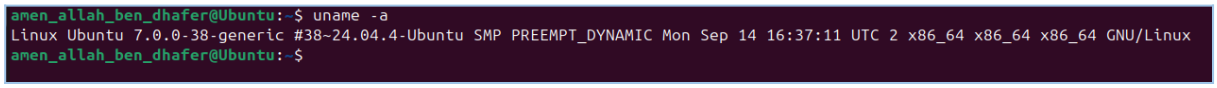


Figure 39 – *Informations système de la machine hôte*

> **Conclusion :** Les conteneurs affichent le **même noyau** (7.0.0-38-generic) que le système hôte. Un conteneur n'embarque pas son propre noyau : il **partage celui de l'hôte** et n'isole que les processus, le système de fichiers et le réseau, ce qui le distingue d'une machine virtuelle.

3. Image Docker hello-world

hello-world est l'exemple de base de Docker : un conteneur qui affiche un message d'accueil puis s'arrête. Il sert à vérifier que la plateforme fonctionne correctement.

```
$ docker run hello-world
```

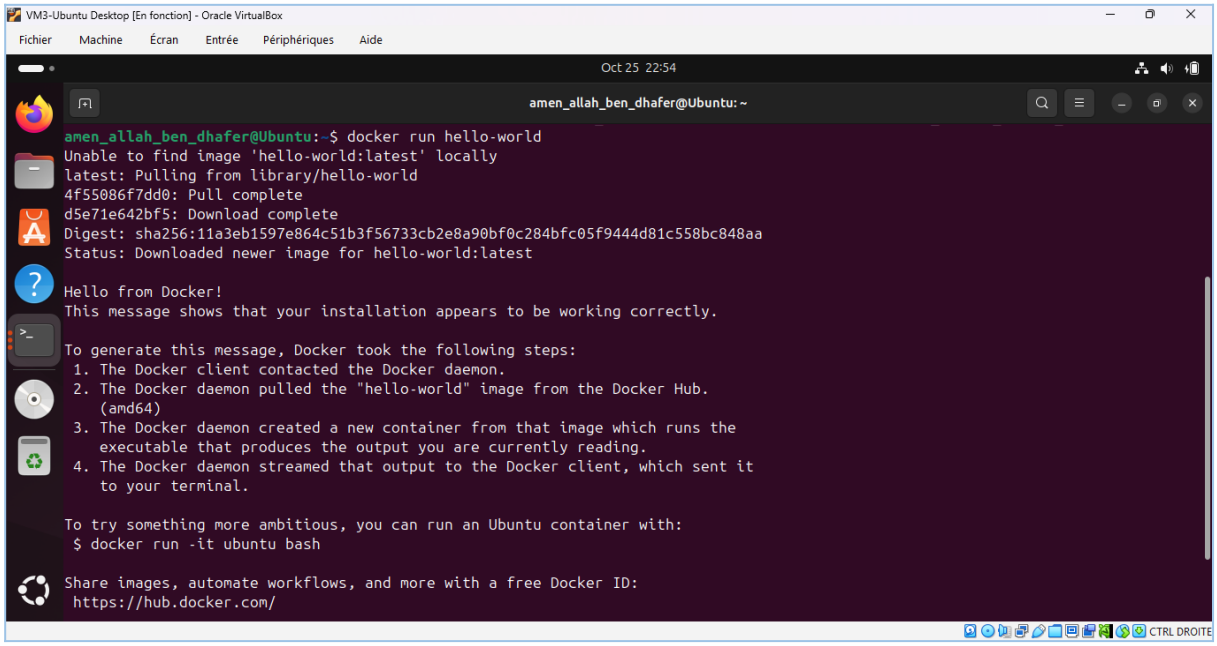


Figure 40 – *Exécution de docker run hello-world*

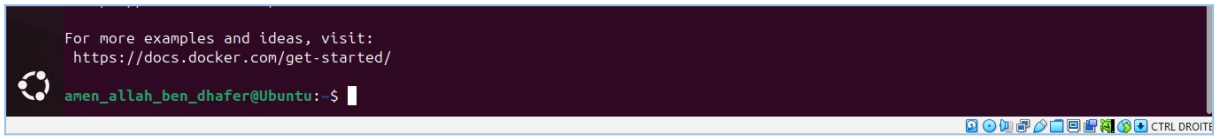


Figure 41 – *Fin du message affiché par le conteneur*

> **Observation :** L'image ayant été supprimée en 2.1.4, Docker la télécharge à nouveau avant de l'exécuter. Le message décrit les quatre étapes : (1) le client Docker contacte le démon ; (2) le démon télécharge l'image `hello-world` depuis Docker Hub (amd64) ; (3) le démon crée un conteneur à partir de l'image, qui exécute le programme produisant le message ; (4) le démon renvoie la sortie au client, qui l'affiche dans le terminal.

3.1 Anatomie de l'image hello-world

La page de l'image sur Docker Hub (`hub.docker.com/_/hello-world`) présente l'image, et son code source est hébergé sur GitHub (`github.com/docker-library/hello-world`).

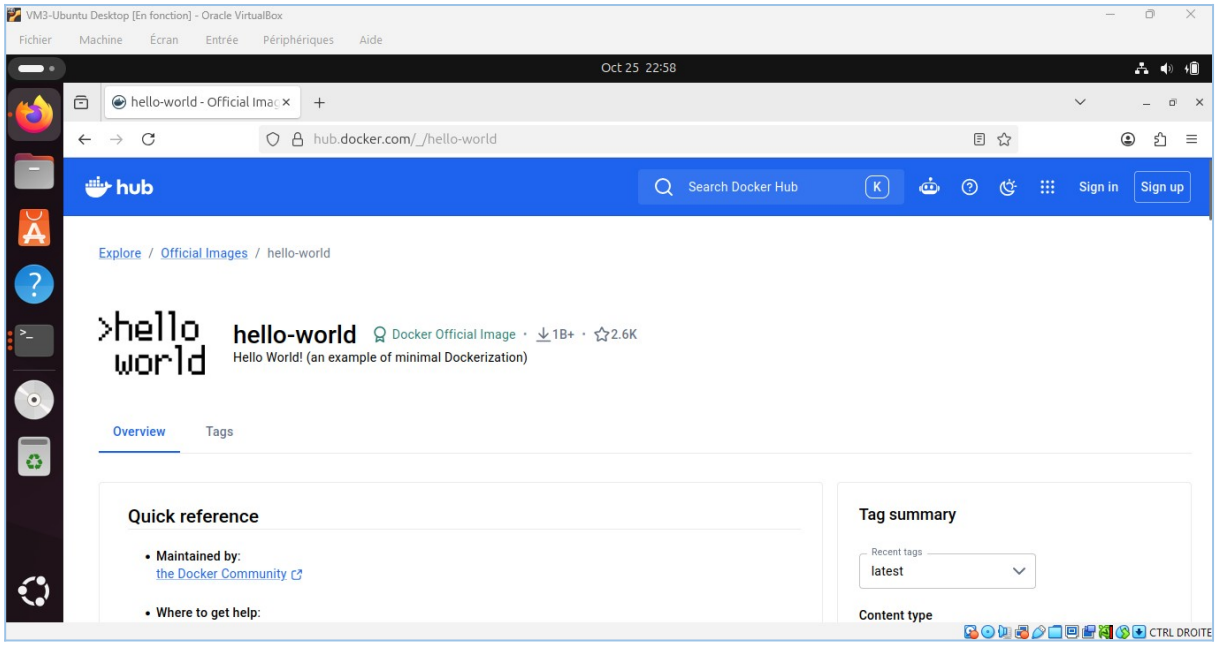


Figure 42 – *Page Docker Hub de l'image officielle hello-world*

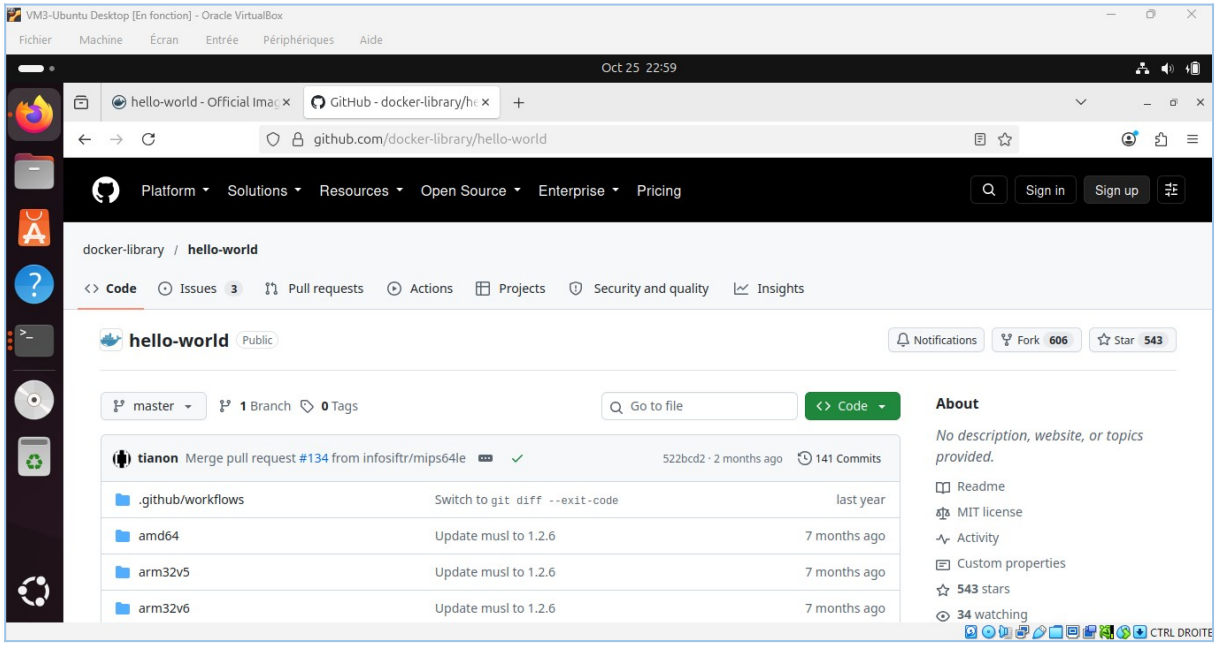


Figure 43 – *Dépôt GitHub docker-library/hello-world (branche master)*

> **Observation :** hello-world est une **image officielle** (« Docker Official Image ») maintenue par la communauté Docker. Le dépôt GitHub contient le code source, les Dockerfile et les binaires par architecture (`amd64`, `arm32v5`, `arm32v6`…).

3.2 Code source hello.c

Sous la branche principale **master**, le fichier source `hello.c` est accessible à la racine du dépôt.

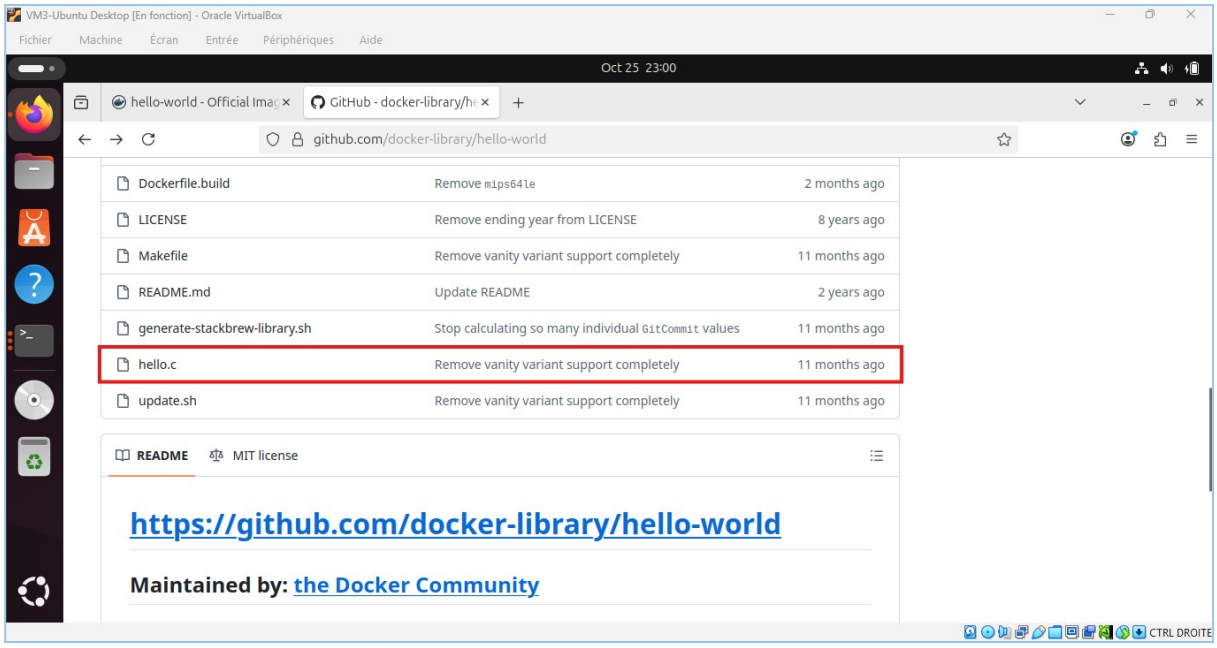


Figure 44 – *Fichier hello.c dans le dépôt*

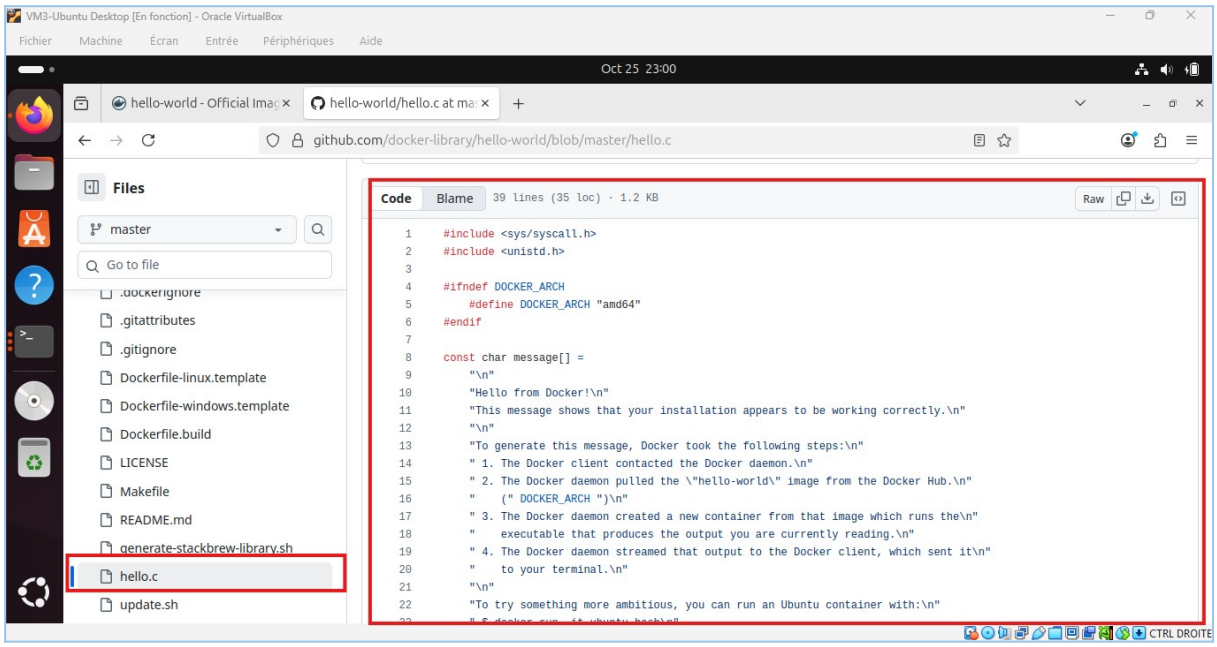


Figure 45 – *Contenu du fichier hello.c*

> **Observation :** Le programme `hello.c` définit le texte du message d'accueil dans un tableau de caractères, puis l'écrit sur la **sortie standard** (le terminal) par un appel système d'écriture. C'est ce message qui s'affiche à l'exécution du conteneur.

3.3 Fichier Dockerfile

Les dossiers `amd64` et `i386` du dépôt contiennent le fichier **Dockerfile** et le fichier objet **hello**, programme exécutable compilé à partir de `hello.c`.

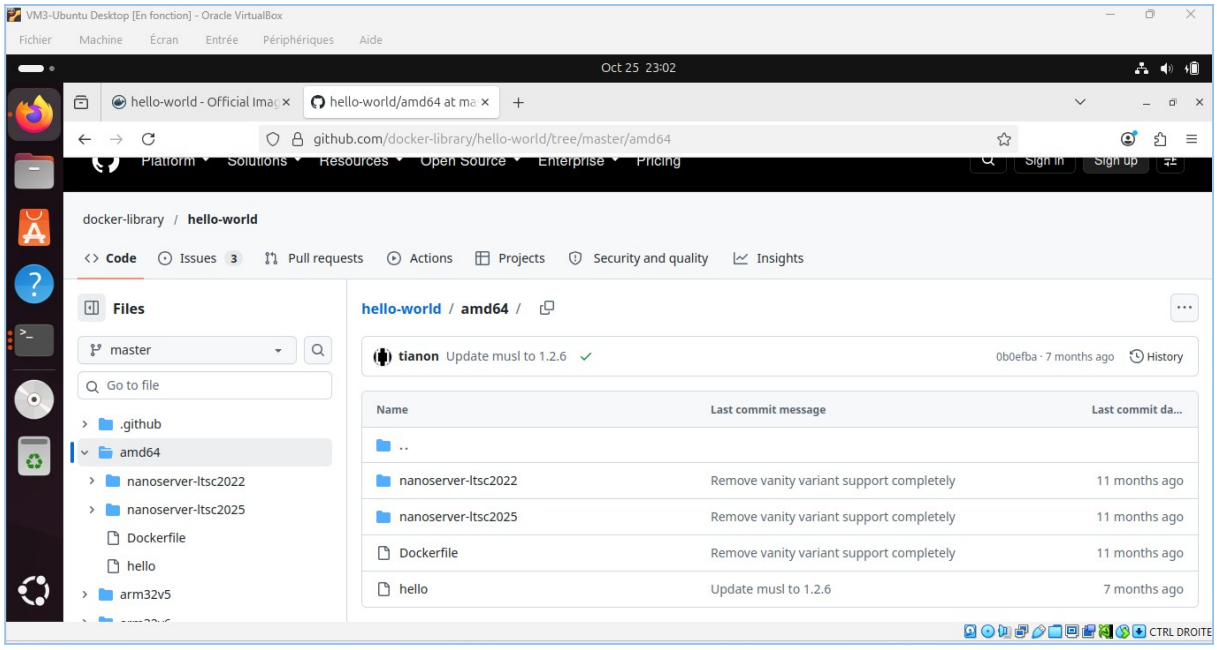


Figure 46 – *Contenu du dossier amd64 : Dockerfile et fichier hello*

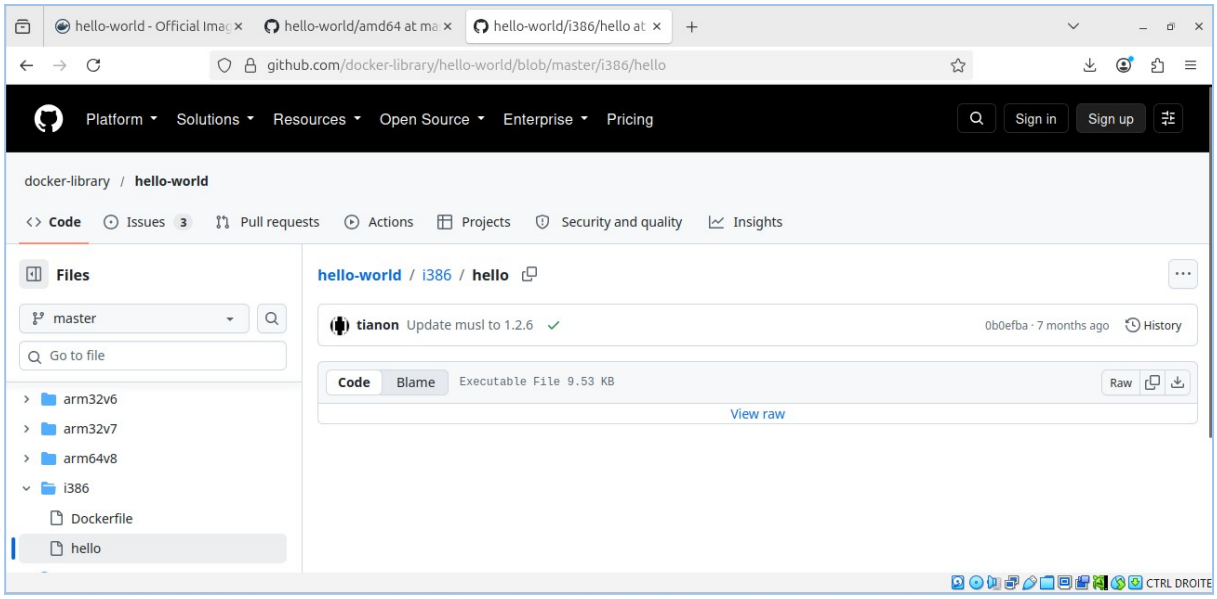


Figure 47 – *Fichier exécutable hello de la branche i386 (9,53 Ko)*

Contenu du Dockerfile

```
$ FROM scratch
$ COPY hello /
$ CMD ["/hello"]
```

Explication : `FROM scratch` : image de base vide, sans système d'exploitation.

`COPY hello /` : copie l'exécutable `hello` à la racine du système de fichiers de l'image.

`CMD ["/hello"]` : commande lancée au démarrage du conteneur, qui exécute le programme `hello`.

L'image est donc minuscule (25,9 kB, figure 19) : elle ne contient que le programme `hello`.

Conclusion

Ce TP a permis d'installer Docker sous Ubuntu, de gérer le cycle de vie des **images** (chercher, télécharger, supprimer) et des **conteneurs** (créer, démarrer, arrêter, supprimer, exécuter une commande), et de comprendre le fonctionnement de l'image **hello-world**. Il a aussi montré que les conteneurs partagent le noyau de la machine hôte.